

Ruta de auto-aprendizaje

Sección 3 — Gramática: vectores de palabras, morfología computacional y generación de cloze

Para dominar por cuenta propia todo lo que usa esta sección: de la similitud coseno al parser de dependencias, con los ejemplos corriendo en el propio repo.

0. Mapa de la ruta

Fase	Tema	Tiempo orientativo	Resultado
1	Vectores de palabras y similitud coseno	1 semana	Calcular a mano y en consola la similitud entre dos palabras
2	Morfología computacional con spaCy	1-2 semanas	Leer el análisis de una frase (POS, morph, dependencias) y localizar huecos
3	El problema del cloze: unicidad vs dificultad	1 semana	Justificar cada filtro de pipeline/gramatica.py y su formalización
4	Implementación y experimentos en el repo	1-2 semanas	Añadir un tema nuevo al generador y defender sus distractores

Regla de oro (la misma de la Sección 2): cada concepto se aprende **tres veces** — en papel (la fórmula), en la consola (el número) y en el repo (el código que lo usa).

Fase 1 — Vectores de palabras y similitud coseno

1.1 La hipótesis distribucional

«Conocerás una palabra por la compañía que lleva» (Firth, 1957): palabras que aparecen en contextos parecidos significan cosas parecidas. Los **word embeddings** llevan esa idea al álgebra: cada palabra es un vector en $\mathbb{R}^d$ (aquí $d = 300$, modelo de_core_news_md) entrenado para que la cercanía geométrica refleje la cercanía distribucional. Consecuencia útil para esta sección: dos formas del mismo paradigma («dem»/«den», «mit»/«bei») viven cerca — son confundibles — y eso es exactamente lo que quieres en un distractor difícil.

1.2 La similitud coseno

$\cos(u, v) = (u \cdot v) / (||u|| \cdot ||v||)$ mide el ángulo entre dos vectores ignorando su longitud: 1 = misma dirección, 0 = ortogonales. Se prefiere al producto escalar crudo porque la norma de un embedding crece con la frecuencia de la palabra, y no quieres que «der» gane todas las comparaciones solo por frecuente. Compruébalo con el modelo real:

```
# PowerShell con PYTHONUTF8=1, en la raíz del repo:
python -c "import spacy; nlp = spacy.load('de_core_news_md');
a, b, c = nlp.vocab['dem'], nlp.vocab['den'], nlp.vocab['Katze'];
print(a.similarity(b)); # alto: mismo paradigma
print(a.similarity(c))" # bajo: nada que ver
```

Dónde vive en el repo: `coseno()` y `ordenar_por_coseno()` en `pipeline/gramatica.py` (numpy puro, sin magia). Ejercicio de 15 minutos: reproduce el ranking de distractores de un ejercicio de declinación de `src/data/gramatica.json` puntuando tú mismo los 5 rivales del artículo.

1.3 Qué estudiar y dónde

Recurso	Qué aporta
Jurafsky & Martin, «Speech and Language Processing» (3ª ed., borrador libre en web.stanford.edu/~jurafsky/slp3), cap. «Vector Semantics and Embeddings»	El capítulo canónico: de la matriz de co-ocurrencias al coseno y word2vec
Mikolov et al. (2013), «Efficient Estimation of Word Representations» (word2vec)	El paper que popularizó los embeddings; legible
Bojanowski et al. (2017), «Enriching Word Vectors with Subword Information» (fastText)	Vectores con sub-palabras: por qué funcionan bien en alemán (compuestos)

Fase 2 — Morfología computacional con spaCy

2.1 Las tres capas del análisis que usa el generador

Capa	Qué es	Cómo la usa gramatica.py
POS (Universal POS tags)	Categoría gramatical universal: DET, ADP, VERB, AUX...	Preselección del token del hueco (DET para artículos, ADP para preposiciones)
Morfología (UD features)	Rasgos Case/Gender/Number/Person/VerbForm del token	La pista del ejercicio y el filtro de concordancia
Dependencias (TIGER)	Árbol sintáctico; en alemán el prefijo separado se marca svp	Detectar verbos separables y el subárbol de la preposición (marca de caso visible)

```
# Mira lo que ve el generador (consola):
python -c "import spacy; nlp = spacy.load('de_core_news_md');
doc = nlp('Maria bereitet das Fruehstueck vor.');"
print([(t.text, t.pos_, t.dep_, str(t.morph)) for t in doc])"
```

Fíjate en «vor»: `dep_ = svp` y `head = «bereitet»` — así reconstruye el pipeline el lema `vorbereiten` (igual que en la Sección 1). Ejercicio: pasa por consola tres frases de una lectura y localiza a mano qué tokens elegiría cada uno de los cuatro temas.

2.2 El parser como oráculo de gramaticalidad

El filtro de conjugación re-parsea la frase con el candidato sustituido y pregunta si la concordancia sobrevive. Es un uso de «parser como oráculo»: no tenemos un juez de gramaticalidad perfecto, pero el analizador morfológico detecta con alta precisión que en «Die Felder war...» el número no cuadra. Limitación honesta (está en `metricas-seccion3.pdf`): hereda los errores de spaCy; por eso el corpus se restringe a frases cortas, donde la morfología es fiable.

2.3 Qué estudiar y dónde

Recurso	Qué aporta
Curso oficial de spaCy (course.spacy.io , gratis, interactivo)	POS, morfología, dependencias y Matcher con ejercicios ejecutables
universaldependencies.org (guía de features)	El estándar de Case/Gender/Number/Person que emite spaCy
Jurafsky & Martin, cap. «Dependency Parsing»	Cómo funciona por dentro un parser de dependencias

Fase 3 — El problema del cloze con respuesta única

Un ejercicio es (c, r, D): contexto con hueco, respuesta y k distractores. Las dos propiedades en tensión (formalizadas en [metricas-seccion3.pdf](#)): **P1 unicidad** — ningún $d \in D$ puede dar una frase válida — y **P2 dificultad** — los d deben ser plausibles. El generador maximiza P2 sujeto a P1: el paradigma define el conjunto, el coseno lo ordena (P2), y un filtro por tema garantiza la agramaticalidad (P1).

3.1 Los cuatro filtros, como razonamiento

Tema	Pregunta que debes saber responder
Declinación	¿Por qué las otras formas del artículo son agramaticales «por construcción»? (el sustantivo fija género/número; la sintaxis, el caso)
Preposición	¿Por qué el pool es el caso OPUESTO y por qué se exige marca de caso visible? (sin «dem/einem/rotem» a la vista, «nach sieben Uhr» también valdría)
Conjugación	¿Por qué «geht» no puede ser distractor de «ging»? (concuerta en persona/número: sería otra frase válida, solo que en presente)
Separables	¿Por qué la respuesta debe formar un verbo atestiguado y los distractores no? (aufmachen vs zumachen: ambos existen → ambigüedad)

Ejercicio de 30 minutos: toma cinco ejercicios generados de `src/data/gramatica.json` e intenta romperlos — busca un distractor que produzca una frase válida. Si encuentras uno, tienes un contraejemplo real que discutir (y un filtro que mejorar): así se audita un generador.

3.2 Contexto en la literatura

Recurso	Qué aporta
Zesch & Melamud (2014), «Automatic Generation of Challenging Distractors Using Context-Sensitive Inference Rules» (BEA workshop)	El problema exacto de esta sección en la literatura de NLP educativo
Susanti et al. (2018), «Automatic distractor generation for multiple-choice English vocabulary questions»	Panorama de estrategias de distractores (WordNet, embeddings, frecuencia)
BEA Workshop (Building Educational Applications, ACL)	La serie donde se publica la generación automática de ejercicios; buscar «cloze» y «distractor»

Fase 4 — La implementación de este repo, por dentro

4.1 Insights de diseño que conviene entender

Insight	Por qué importa
Todo el trabajo vectorial ocurre offline	El frontend consume JSON estático por dynamic import; ni spaCy ni numpy en el bundle
El paradigma define el conjunto; el coseno solo ordena	Cada método hace lo que hace bien: sin paradigma hay distractores de otra categoría; sin coseno, distractores fáciles
Selección estratificada round-robin por fuente	Sin ella, la primera lectura en orden alfabético acaparaba el tope de 40 ejercicios
Clave de progreso estable (fuente antes respuesta), no el id	El progreso del usuario sobrevive a regenerar el corpus, que renumera los ids
Determinismo de punta a punta	Mismo corpus → mismo JSON; misma semilla → mismas opciones; auditable y testeable
El LCG de board.js se normaliza a [0,1) en el motor de gramática	El generador compartido tenía un borde (hash negativo → valores negativos) que rompería el barajado; se corrige sin tocar los tableros del juego

4.2 Paquetería y herramientas

Herramienta	Rol en la sección
spaCy + de_core_news_md	POS, morfología, dependencias y vectores (20.000 × 300)
numpy	El coseno a mano (norma y producto escalar); sin dependencias extra
Vitest 4	Tests del motor puro (opciones, agrupación por lectura, progreso)
ReportLab	Genera estos PDF (docs/generar_*.py)
Python 3 + PYTHONUTF8=1	pipeline/gramatica.py y los generadores (regla: ensure_ascii=False para los umlauts)
localStorage	Persistencia sin backend: gramatica.hechos.v1

4.3 Experimentos propuestos (en orden de dificultad)

1) Cambia `k` (`N_DISTRACTORES`) a 5 y regenera: ¿cuántos ejercicios de conjugación sobreviven al filtro de concordancia con 5 rivales? 2) Invierte el orden del coseno (quedarte con los MENOS parecidos) y compara la dificultad percibida de una sesión: es la forma más rápida de sentir qué aporta el ranking. 3) Mide la precisión del oráculo: toma 20 distractores de conjugación, júzgalos a mano y estima cuántos son realmente agramaticales. 4) Añade un tema nuevo al generador (p. ej. verbo en 2ª posición: detecta la frase declarativa y ofrece órdenes de palabras alternativos como distractores) con su filtro de unicidad. 5) Conecta gramática con repaso: al fallar un ejercicio, mete la palabra del hueco en la bolsa (Sección 2) para que el SM-2 la programe.

5. Recursos externos (lista corta y buena)

Recurso	Tipo	Para qué
Jurafsky & Martin, «Speech and Language Processing» 3ª ed. (borrador libre)	Libro	Vector semantics + dependency parsing: las dos patas de la sección
course.spacy.io	Curso interactivo	spaCy de cero a Matcher en unas horas
universaldependencies.org	Estándar	Los rasgos morfológicos que emite spaCy
Zesch & Melamud (2014), BEA workshop	Paper	Distractores difíciles con garantía de incorrección: el antecedente directo del método híbrido
Duden Grammatik (o canoo.net/Grammis del IDS Mannheim, gratis)	Referencia	La gramática alemana de verdad: declinación, rección y separables

6. Lista de verificación de dominio

Puedes dar la sección por dominada cuando, sin mirar el código: (a) calculas el coseno de dos vectores pequeños a mano y explicas por qué se normaliza; (b) lees el análisis spaCy de una frase alemana y localizas el hueco que elegiría cada tema; (c) enuncias P1/P2 y el filtro de unicidad de cada tema con su contraejemplo evitado («um sieben Uhr», `ging`→`geht`, `aufmachen/zumachen`); (d) explicas por qué la selección es estratificada y qué pasaba sin ella; (e) propones un tema nuevo con su paradigma, su pool de distractores y su filtro — y lo implementas en `pipeline/gramatica.py`.